ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅԱՆ ԿՐԹՈՒԹՅԱՆ ԵՎ ԳԻՏՈՒԹՅԱՆ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ

ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆԻ ԳՅՈՒՄՐՈՒ ՄԱՍՆԱՃՅՈՒՂ

- **Առարկա՝** Օբյեկտ կողմնորոշված ծրագրավորում
- **Խումբ՝** ԳԻԿ 519Բ
- **Կուրս՝** 2-րդ
- **Ուսանող՝** Գաբրիել Քոչարյան
- **Դասախոս՝** Սոնյա Սահակյան
- **ԳՅՈՒՄՐԻ – 2026**

Լաբորատոր աշխատանք 1

Երկու թվերի գործողություններ

- **Աշխատանքի նպատակը**: Ծանոթանալ C++ ծրագրավորման լեզվում ամբողջ թվերի մուտքագրման, ստուգման և թվաբանական գործողությունների կատարման եղանակներին: Զարգացնել սխալ մուտքային տվյալների հայտնաբերման և մշակման հմտությունները՝ օգտագործելով պայմաններ և ցիկլեր:
- **Խնդրի պահանջը**: Մուտքագրել երկու ամբողջ դրական թիվ: Ստուգել մուտքագրված արժեքների ճիշտ լինելը և արտածել դրանց գումարը, տարբերությունը, արտադրյալը և քանորդը: Տառերի, այլ նշանների, բացասական թվերի կամ զրոյի մուտքագրման դեպքում արտածել համապատասխան սխալի հաղորդագրություն և կրկին պահանջել մուտքագրել ճիշտ արժեք: Բաժանման արդյունքը պահել որպես ամբողջ թիվ (մնացորդը հաշվել անհրաժեշտ չէ):

Ծրագրի կոդը (`lab1.cpp`):

```
#include <iostream>

#include <limits>

using namespace std;

int getPositiveInteger(const string& prompt) {
```

```
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            if (value > 0) {
                return value;
            } else {
                cout << "Սխալ մուտք. թիվը պետք է լինի 0-ից մեծ (դրական):" << endl;
            }
        } else {
            cout << "Սխալ մուտք. մուտքագրեք ամբողջ թիվ:" << endl;
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
}
int main() {
    int firstNumber = getPositiveInteger("Մուտքագրեք առաջին ամբողջ դրական թիվը: ");
    int secondNumber = getPositiveInteger("Մուտքագրեք երկրորդ ամբողջ դրական թիվը: ");
    int sum = firstNumber + secondNumber;
    int difference = firstNumber - secondNumber;
    int product = firstNumber * secondNumber;
```

```
    int quotient = firstNumber / secondNumber;

    cout << "\n--- Արդյունքներ ---" << endl;

    cout << "Գումար: " << sum << endl;

    cout << "Տարբերություն: " << difference << endl;

    cout << "Արտադրյալ: " << product << endl;

    cout << "Քանորդ: " << quotient << endl;

    return 0;

}
```

Լաբորատոր աշխատանք 2

Նշված բառի հեռացում

- **Աշխատանքի նպատակը**: Զարգացնել C++ լեզվում տողերի հետ աշխատելու և ստանդարտ գրադարանային ֆունկցիաներ կիրառելու հմտությունները:
- **Խնդրի պահանջը**: Մուտքագրել նախադասություն և հեռացվող բառ: Կազմել նոր տող՝ բաց թողնելով այդ բառի բոլոր հանդիպումները, իսկ մնացած բառերը միացնել մեկական բացատով:
- **Կիրառվող ֆունկցիաներ՝** `strtok`, `strcmp`, `strcat`:

Ծրագրի կոդը (`lab2.cpp`):

```
#include <iostream>

#include <cstring>

using namespace std;

int main() {

    char sentence[500];

    char removeWord[100];
```

```
    char result[500] = "";
    cout << "Մուտքագրեք նախադասությունը: ";
    cin.getline(sentence, 500);
    cout << "Մուտքագրեք հեռացվող բառը: ";
    cin >> removeWord;
    char* token = strtok(sentence, " ");
    bool first = true;
    while (token != nullptr) {
        if (strcmp(token, removeWord) != 0) {
            if (!first) {
                strcat(result, " ");
            }
            strcat(result, token);
            first = false;
        }
        token = strtok(nullptr, " ");
    }
    cout << "Արդյունք: " << result << endl;
    return 0;
}
```

Լաբորատոր աշխատանք 3

17. Դինամիկ գրանցումների շտեմարան

- **Աշխատանքի նպատակը**: Ամրապնդել դինամիկ զանգվածների, հիշողության կառավարման (`new`, `delete[]`, `nullptr`) և ցուցիչային աշխատանքի գործնական հմտությունները:
- **Խնդրի պահանջը**: Օգտվողից ստանալ ապրանքների քանակը և `new`-ով ստեղծել `Product` կառուցվածքների դինամիկ զանգված: Մուտքագրել տվյալները, ֆունկցիայի միջոցով գտնել միջին գնից բարձր ապրանքները, փոխել ընտրված ապրանքի քանակը ցուցիչով և վերջում ճիշտ ազատել հիշողությունը:

Ծրագրի կոդը (`lab3.cpp`):

```
#include <iostream>

#include <string>

using namespace std;

struct Product {

    string name;

    double price;

    int quantity;

};

void printAboveAverage(const Product* products, int count, double avg) {

    cout << "\nՄիջին գնից (" << avg << ") բարձր ապրանքները:" << endl;

    for (int i = 0; i < count; i++) {

        if ((products + i)->price > avg) {

            cout << "- " << (products + i)->name << " | Գին: " << (products + i)->price

                << " | Քանակ: " << (products + i)->quantity << endl;
```

```
        }
    }
}
void changeQuantity(Product* ptr, int newQty) {
    if (ptr != nullptr) {
        ptr->quantity = newQty;
    }
}
int main() {
    int n;
    cout << "Մուտքագրեք ապրանքների քանակը: ";
    cin >> n;
    Product* products = new Product[n];
    double totalPrice = 0;
    for (int i = 0; i < n; i++) {
        cout << "Ապրանք " << i + 1 << " (անուն, գին, քանակ): ";
        cin >> products[i].name >> products[i].price >> products[i].quantity;
        totalPrice += products[i].price;
    }
    double avgPrice = (n > 0) ? (totalPrice / n) : 0;
    printAboveAverage(products, n, avgPrice);
    if (n > 0) {
```

```
        int index, newQty;

        cout << "\nԸնտրեք ապրանքի ինդեքսը (0-ից " << n - 1 << ") քանակը փոխելու համար:
";

        cin >> index;

        if (index >= 0 && index < n) {

            cout << "Մուտքագրեք նոր քանակը: ";

            cin >> newQty;

            changeQuantity(&products[index], newQty);

            cout << "Քանակը թարմացվեց: " << products[index].name << " -> " <<
products[index].quantity << endl;

        }

    }

    delete[] products;

    products = nullptr;

    return 0;

}
```

Լաբորատոր աշխատանք 4

N17. Հիվանդների գրանցում

- **Աշխատանքի նպատակը**: Յուրացնել բարդ կառուցվածքների (`struct`) և ֆունկցիոնալ մշակման ալգորիթմները:
- **Խնդրի պահանջը**: Ստեղծել `Patient` կառուցվածք՝ `name`, `surname`, `diagnosis` և `age` դաշտերով: Մուտքագրել 8 հիվանդի տվյալներ և ֆունկցիաների միջոցով՝
    1. Արտածել բոլոր հիվանդների տվյալները:
    2. Գտնել ամենատարեց հիվանդին:

3. Արտածել տրված ախտորոշմամբ հիվանդներին:
4. Հաշվել 18 տարեկանից փոքր հիվանդների քանակը:

Ծրագրի կոդը (`lab4.cpp`):

```
#include <iostream>

#include <string>

using namespace std;

struct Patient {

    string name;

    string surname;

    string diagnosis;

    int age;

};

void printAllPatients(const Patient patients[], int count) {

    cout << "\n--- Բոլոր հիվանդների ցուցակը ---" << endl;

    for (int i = 0; i < count; i++) {

        cout << i + 1 << ". " << patients[i].name << " " << patients[i].surname

            << " | Ախտորոշում: " << patients[i].diagnosis << " | Տարիք: " << patients[i].age << endl;

    }

}

void findOldestPatient(const Patient patients[], int count) {

    if (count <= 0) return;

    int oldestIdx = 0;
```

```
    for (int i = 1; i < count; i++) {
        if (patients[i].age > patients[oldestIdx].age) {
            oldestIdx = i;
        }
    }
    cout << "\nԱմենատարեց հիվանդը: " << patients[oldestIdx].name << " " << patients[oldestIdx].surname
         << " (" << patients[oldestIdx].age << " տարեկան)" << endl;
}
void printPatientsByDiagnosis(const Patient patients[], int count, const string& diag) {
    cout << "\n--- Հիվանդներ " << diag << " ախտորոշմամբ ---" << endl;
    bool found = false;
    for (int i = 0; i < count; i++) {
        if (patients[i].diagnosis == diag) {
            cout << "- " << patients[i].name << " " << patients[i].surname << endl;
            found = true;
        }
    }
    if (!found) cout << "Այս ախտորոշմամբ հիվանդ չկա:" << endl;
}
int countUnder18(const Patient patients[], int count) {
    int cnt = 0;
```

```
    for (int i = 0; i < count; i++) {
        if (patients[i].age < 18) cnt++;
    }
    return cnt;
}
int main() {
    const int COUNT = 8;
    Patient patients[COUNT];
    cout << "Մուտքագրեք " << COUNT << " հիվանդի տվյալներ (անուն, ազգանուն, ախտորոշում, տարիք):" << endl;
    for (int i = 0; i < COUNT; i++) {
        cout << "Հիվանդ " << i + 1 << ": ";
        cin >> patients[i].name >> patients[i].surname >> patients[i].diagnosis >> patients[i].age;
    }
    printAllPatients(patients, COUNT);
    findOldestPatient(patients, COUNT);
    string searchDiag;
    cout << "\nՄուտքագրեք որոնվող ախտորոշումը: ";
    cin >> searchDiag;
    printPatientsByDiagnosis(patients, COUNT, searchDiag);
    cout << "\n18 տարեկանից փոքր հիվանդների քանակը: " << countUnder18(patients, COUNT) << endl;
    return 0;
```

}

Լաբորատոր աշխատանք 5

N17. Բեռնատարների փոխադրումների հաշվառում

- **Աշխատանքի նպատակը**: Կիրառել կառուցվածքների զանգվածներ բեռնափոխադրումների վիճակագրական և վերլուծական խնդիրներ լուծելու համար:
- **Խնդրի պահանջը**: Ստեղծել `Cargo` կառուցվածք՝ `truckNumber`, `destination`, `cargoWeight`, `maxWeight`, `distance` դաշտերով: Մուտքագրել 10 փոխադրում և ֆունկցիաների միջոցով՝
    1. Հաշվել յուրաքանչյուր բեռնատարի բեռնվածության տոկոսը:
    2. Գտնել ամենաբարձր բեռնվածություն ունեցող մեքենան:
    3. Արտածել 90%-ից ավելի բեռնված մեքենաները:
    4. Հաշվել տրված `destination` ուղղությամբ տեղափոխվող բեռի ընդհանուր քաշը:
    5. Հայտնաբերել `maxWeight`-ը գերազանցող գրանցումները:

Ծրագրի կոդը (`lab5.cpp`):

```
#include <iostream>

#include <string>

using namespace std;

struct Cargo {

    string truckNumber;

    string destination;

    double cargoWeight;

    double maxWeight;

    double distance;

};
```

```
double calculateLoadPercentage(const Cargo& c) {
    return (c.maxWeight > 0) ? (c.cargoWeight / c.maxWeight * 100.0) : 0.0;
}
void printHighestLoaded(const Cargo cargos[], int count) {
    if (count <= 0) return;
    int maxIdx = 0;
    double maxPct = calculateLoadPercentage(cargos[0]);
    for (int i = 1; i < count; i++) {
        double pct = calculateLoadPercentage(cargos[i]);
        if (pct > maxPct) {
            maxPct = pct;
            maxIdx = i;
        }
    }
    cout << "\nԱմենաբարձր բեռնվածությամբ մեքենան: " << cargos[maxIdx].truckNumber
         << " (" << maxPct << "%)" << endl;
}
void printOver90Percent(const Cargo cargos[], int count) {
    cout << "\n--- 90%-ից ավելի բեռնված մեքենաները ---" << endl;
    for (int i = 0; i < count; i++) {
        double pct = calculateLoadPercentage(cargos[i]);
```

```
        if (pct > 90.0) {
            cout << cargos[i].truckNumber << " -> " << pct << "%" << endl;
        }
    }
}
double totalWeightByDestination(const Cargo cargos[], int count, const string& dest) {
    double total = 0;
    for (int i = 0; i < count; i++) {
        if (cargos[i].destination == dest) {
            total += cargos[i].cargoWeight;
        }
    }
    return total;
}
void checkExceededMaxWeight(const Cargo cargos[], int count) {
    cout << "\n--- MaxWeight-ը գերազանցող գրանցումները ---" << endl;
    bool found = false;
    for (int i = 0; i < count; i++) {
        if (cargos[i].cargoWeight > cargos[i].maxWeight) {
            cout << "Մեքենա " << cargos[i].truckNumber << ": քաշ=" << cargos[i].cargoWeight
                << ", առավելագույն=" << cargos[i].maxWeight << endl;
            found = true;
```

```
        }

    }

    if (!found) cout << "Գերազանցող գրանցումներ չկան:" << endl;

}

int main() {

    const int COUNT = 10;

    Cargo cargos[COUNT];

    cout << "Մուտքագրեք " << COUNT << " փոխադրման տվյալներ (մեքենայի համար, ուղղություն, բեռի քաշ, maxWeight, հեռավորություն):" << endl;

    for (int i = 0; i < COUNT; i++) {

        cout << "Փոխադրում " << i + 1 << ": ";

        cin >> cargos[i].truckNumber >> cargos[i].destination >> cargos[i].cargoWeight >> cargos[i].maxWeight >> cargos[i].distance;

    }

    cout << "\n--- Բեռնվածության տոկոսները ---" << endl;

    for (int i = 0; i < COUNT; i++) {

        cout << cargos[i].truckNumber << ": " << calculateLoadPercentage(cargos[i]) << "%" << endl;

    }

    printHighestLoaded(cargos, COUNT);

    printOver90Percent(cargos, COUNT);

    string targetDest;

    cout << "\nՄուտքագրեք ուղղությունը (destination) ընդհանուր քաշը հաշվելու համար: ";
```

```
    cin >> targetDest;

    cout << targetDest << " ուղղությամբ ընդհանուր բեռի քաշը: "

        << totalWeightByDestination(cargos, COUNT, targetDest) << " կգ" << endl;

    checkExceededMaxWeight(cargos, COUNT);

    return 0;

}
```

Լաբորատոր աշխատանք 6

N17. Ֆայլերի պահուստավորման հերթ և ծավալ

- **Աշխատանքի նպատակը**: Ուսումնասիրել C++ դասերի (`class`), օբյեկտների, մեթոդների և տվյալների վավերացման սկզբունքները:
- **Խնդրի պահանջը**: Ստեղծել `BackupJob` դաս՝ `id`, `fileName`, `sizeMB`, `priority`, `completed` դաշտերով: Ստեղծել 12 պահուստավորման առաջադրանք և իրականացնել հետևյալ մեթոդները.
    1. Կոնստրուկտորով ստուգել ծավալը և առաջնահերթությունը (1–5):
    2. `nextJob()` մեթոդով ընտրել ամենաբարձր առաջնահերթությամբ չավարտված առաջադրանքը (հավասարության դեպքում՝ փոքր ծավալովը):
    3. `completeJob()` մեթոդով նշել առաջադրանքը ավարտված՝ կանխելով կրկնակի կատարումը:
    4. Հաշվել պահուստավորման համար անհրաժեշտ ժամանակը՝ տրված փոխանցման արագությամբ:
    5. Գտնել, թե որ առաջադրանքները կհասցնեն ավարտվել տրված ժամանակային սահմանում:

Ծրագրի կոդը (`lab6.cpp`):

```
#include <iostream>

#include <string>

#include <vector>

#include <algorithm>
```

```
using namespace std;

class BackupJob {

private:

    int id;

    string fileName;

    double sizeMB;

    int priority;

    bool completed;

public:

    BackupJob(int jobId, string name, double size, int prio) {

        id = jobId;

        fileName = name;

        sizeMB = (size > 0) ? size : 1.0;

        priority = (prio >= 1 && prio <= 5) ? prio : 1;

        completed = false;

    }

    int getId() const { return id; }

    string getFileName() const { return fileName; }

    double getSizeMB() const { return sizeMB; }

    int getPriority() const { return priority; }

    bool isCompleted() const { return completed; }
```

```
    void completeJob() {
        if (!completed) {
            completed = true;
            cout << "Առաջադրանք #" << id << " (" << fileName << ") հաջողությամբ ավարտվեց:"
<< endl;
        } else {
            cout << "Առաջադրանք #" << id << "-ն արդեն ավարտված է:" << endl;
        }
    }
};
int nextJob(const vector<BackupJob>& jobs) {
    int bestIdx = -1;
    for (size_t i = 0; i < jobs.size(); i++) {
        if (!jobs[i].isCompleted()) {
            if (bestIdx == -1) {
                bestIdx = i;
            } else {
                if (jobs[i].getPriority() > jobs[bestIdx].getPriority()) {
                    bestIdx = i;
                } else if (jobs[i].getPriority() == jobs[bestIdx].getPriority()) {
                    if (jobs[i].getSizeMB() < jobs[bestIdx].getSizeMB()) {
                        bestIdx = i;
```

```
                }
            }
        }
    }
    return bestIdx;
}
double calculateTime(double sizeMB, double speedMBps) {
    return (speedMBps > 0) ? (sizeMB / speedMBps) : 0;
}
int main() {
    vector<BackupJob> jobs;
    jobs.push_back(BackupJob(1, "doc1.pdf", 120, 3));
    jobs.push_back(BackupJob(2, "image1.png", 50, 5));
    jobs.push_back(BackupJob(3, "db.sqlite", 500, 5));
    jobs.push_back(BackupJob(4, "video.mp4", 800, 2));
    jobs.push_back(BackupJob(5, "notes.txt", 10, 1));
    jobs.push_back(BackupJob(6, "backup1.zip", 300, 4));
    jobs.push_back(BackupJob(7, "code.cpp", 5, 5));
    jobs.push_back(BackupJob(8, "system.iso", 1200, 3));
    jobs.push_back(BackupJob(9, "data.csv", 45, 2));
    jobs.push_back(BackupJob(10, "music.mp3", 80, 1));
```

```
    jobs.push_back(BackupJob(11, "presentation.pptx", 150, 4));

    jobs.push_back(BackupJob(12, "logs.log", 25, 3));

    int idx = nextJob(jobs);

    if (idx != -1) {

        cout << "Հաջորդ կատարվող առաջադրանքը: #" << jobs[idx].getId() << " (" <<
jobs[idx].getFileName() << ")" << endl;

        jobs[idx].completeJob();

    }

    double speed = 10.0; // MB/sec

    cout << "\n--- Առաջադրանքների կատարման ժամանակը (արագություն = " << speed <<
" MB/s) ---" << endl;

    for (const auto& job : jobs) {

        cout << job.getFileName() << " (" << job.getSizeMB() << " MB) -> "

             << calculateTime(job.getSizeMB(), speed) << " վրկ" << endl;

    }

    double timeLimit = 30.0; // վայրկյան

    cout << "\n" << timeLimit << " վայրկյանում ավարտվող առաջադրանքները:" << endl;

    for (const auto& job : jobs) {

        if (calculateTime(job.getSizeMB(), speed) <= timeLimit) {

            cout << "- " << job.getFileName() << endl;

        }

    }
```

```
    return 0;

}
```

Լաբորատոր աշխատանք 7

N17. Տուփերի տեղափոխում

- **Աշխատանքի նպատակը**: Վարժվել կոնստրուկտորների վերբեռնման և օբյեկտների կառուցման մեթոդիկային:
- **Խնդրի պահանջը**: Ստեղծել `Delivery` դաս՝ `boxCount` (`int`) և `weightPerBox` (`double`) դաշտերով:
    1. Առանց պարամետրերի կոնստրուկտորով դաշտերին վերագրել `0` և `0`:
    2. Մեկ պարամետրով կոնստրուկտորով `boxCount`-ին տալ փոխանցված արժեքը, իսկ `weightPerBox`-ին՝ `2`:
    3. Երկու պարամետրով կոնստրուկտորով դաշտերին վերագրել փոխանցված արժեքները:
    - Ավելացնել `calculate()` մեթոդ, որը վերադարձնում է ընդհանուր քաշը (`boxCount * weightPerBox`), և `show()` մեթոդ՝ դաշտերի արժեքները արտածելու համար: `main()` ֆունկցիայում ստեղծել երեք օբյեկտ (յուրաքանչյուր կոնստրուկտորով մեկական), արտածել դրանց տվյալներն ու հաշվարկված արդյունքները:

Ծրագրի կոդը (`lab7.cpp`):

```
#include <iostream>

using namespace std;

class Delivery {

private:

    int boxCount;

    double weightPerBox;

public:

    Delivery() {
```

```
        boxCount = 0;
        weightPerBox = 0.0;
    }
    Delivery(int count) {
        boxCount = count;
        weightPerBox = 2.0;
    }
    Delivery(int count, double weight) {
        boxCount = count;
        weightPerBox = weight;
    }
    double calculate() const {
        return boxCount * weightPerBox;
    }
    void show() const {
        cout << "boxCount: " << boxCount << ", weightPerBox: " << weightPerBox << endl;
    }
};
int main() {
    Delivery d1;
    int c2;
    cout << "Մուտքագրեք օբյեկտ 2-ի boxCount-ը: ";
```

```
    cin >> c2;
    Delivery d2(c2);
    int c3;
    double w3;
    cout << "Մուտքագրեք օբյեկտ 3-ի boxCount-ը և weightPerBox-ը: ";
    cin >> c3 >> w3;
    Delivery d3(c3, w3);
    cout << "\n--- Օբյեկտ 1 ---" << endl;
    d1.show();
    cout << "Ընդհանուր քաշ: " << d1.calculate() << endl;
    cout << "\n--- Օբյեկտ 2 ---" << endl;
    d2.show();
    cout << "Ընդհանուր քաշ: " << d2.calculate() << endl;
    cout << "\n--- Օբյեկտ 3 ---" << endl;
    d3.show();
    cout << "Ընդհանուր քաշ: " << d3.calculate() << endl;
    return 0;
}
```

Լաբորատոր աշխատանք 8

N17. Հողի պարարտացում

- **Աշխատանքի նպատակը**: Զարգացնել օբյեկտ կողմնորոշված ծրագրավորման առաջադեմ հմտությունները (ներառյալ զանգվածների մշակումը և բարդ հաշվարկային մեթոդները):
- **Խնդրի պահանջը**: Ստեղծել `FertilizerPlan` դաս: Պահպանվող տվյալներն են՝ հողամասի կոդը, մակերեսը (հա-ով), մեկ հա-ի համար պարարտանյութի նորման (կգ-ով) և պաշարը (կգ-ով):
    1. **Կոնստրուկտորներ**. `FertilizerPlan()` (ընդհանուր կանոններով), երեք պարամետրով՝ կոդ, մակերես, նորմա (պաշարը՝ 0), չորս պարամետրով՝ բոլոր տվյալները:
    2. **Ստուգումներ**. Մակերեսը և նորման՝ դրական, պաշարը՝ ոչ բացասական: Քանակները և օրերը՝ ամբողջ թվեր, չափումները, գներն ու տոկոսները՝ `double`:
    3. **Փոփոխող մեթոդ**. Ավելացնել դրական չափով պարարտանյութի պաշար:
    4. **Հաշվարկային մեթոդներ**. Հաշվել պահանջվող քանակը և պակասը՝ $\max(0, \text{պահանջվող քանակ} - \text{պաշար})$:
    5. **Զանգվածի մշակում**. Ցուցադրել ամենամեծ պակաս ունեցող հողամասը և բոլոր պակասների գումարը:
    6. **Ստուգիչ օրինակ**. Մուտք՝ մակերես՝ 3, նորմա՝ 80, պաշար՝ 150 $\rightarrow$ Ակնկալվող արդյունք՝ պահանջվող քանակը՝ 240 կգ, պակասը՝ 90 կգ (մինչև փոփոխող մեթոդի կանչը):

Ծրագրի կոդը (`lab8.cpp`):

```
#include <iostream>

#include <string>

#include <algorithm>

#include <vector>

using namespace std;

class FertilizerPlan {

private:

    string code;
```

```
    double area;          // hա
    double normPerHa;     // կգ/hա
    double stock;         // կգ
public:
    FertilizerPlan() : code("DEFAULT"), area(1.0), normPerHa(10.0), stock(0.0) {}
    FertilizerPlan(string c, double a, double n) : code(c), stock(0.0) {
        area = (a > 0) ? a : 1.0;
        normPerHa = (n > 0) ? n : 1.0;
    }
    FertilizerPlan(string c, double a, double n, double s) : code(c) {
        area = (a > 0) ? a : 1.0;
        normPerHa = (n > 0) ? n : 1.0;
        stock = (s >= 0) ? s : 0.0;
    }
    void addStock(double amount) {
        if (amount > 0) {
            stock += amount;
        }
    }
    double requiredAmount() const {
        return area * normPerHa;
```

```
    }

    double shortage() const {

        return max(0.0, requiredAmount() - stock);

    }

    string getCode() const { return code; }

    void showInfo() const {

        cout << "Կոդ: " << code << " | Մակերես: " << area << " հա | Նորմա: " << normPerHa

             << " կգ/հա | Պաշար: " << stock << " կգ | Պահանջվող: " << requiredAmount()

             << " կգ | Պակաս: " << shortage() << " կգ" << endl;

    }

};

int main() {

    cout << "=== Ստուգիչ օրինակ ===" << endl;

    FertilizerPlan sample("SAMP01", 3.0, 80.0, 150.0);

    cout << "Պահանջվող քանակ: " << sample.requiredAmount() << " կգ (Ակնկալվող: 240 կգ)"
<< endl;

    cout << "Պակաս: " << sample.shortage() << " կգ (Ակնկալվող: 90 կգ)" << endl;

    cout << "\n=== Զանգվածի մշակում ===" << endl;

    vector<FertilizerPlan> plans;

    plans.push_back(FertilizerPlan("LAND01", 3.0, 80.0, 150.0));

    plans.push_back(FertilizerPlan("LAND02", 5.0, 100.0, 200.0));

    plans.push_back(FertilizerPlan("LAND03", 2.0, 50.0, 120.0));
```

```
    plans.push_back(FertilizerPlan("LAND04", 10.0, 90.0, 300.0));
    double totalShortage = 0;
    int maxShortageIdx = 0;
    for (size_t i = 0; i < plans.size(); i++) {
        plans[i].showInfo();
        double sh = plans[i].shortage();
        totalShortage += sh;
        if (sh > plans[maxShortageIdx].shortage()) {
            maxShortageIdx = i;
        }
    }
    cout << "\nԱմենամեծ պակասն ունեցող հողամասը: " << plans[maxShortageIdx].getCode()
         << " (" << plans[maxShortageIdx].shortage() << " կգ)" << endl;
    cout << "Բոլոր հողամասերի ընդհանուր պակասը: " << totalShortage << " կգ" << endl;
    return 0;
}
```